Chapter 6

RNA m^5C

6.1 Abstract

The detection of RNA 5-methylcytosine (m^5C) modifications is a critical task in molecular biology, with significant implications for understanding gene expression, RNA stability, and protein interactions. However, the accurate identification of m^5C sites using high-throughput sequencing (HTS) data remains challenging due to high false-positive rates and computational inefficiencies in existing workflows. m^5C-UBSseq is an effective workflow for the accurate detection of m^5C sites. However, the current workflow faces challenges in terms of poor I/O performance and scalability during actual execution. To address this, we analyzed the workflow using a combination of software instrumentation and the Intel VTune Profiler and proposed effective optimization strategies.

Our optimizations are initially applied to the entire workflow and then focused on individual tools, which include: (1) parallelizing the independent steps within the workflow; (2) transitioning the working environment to in-memory filesystem to significantly enhance I/O performance, particularly for large file operations; (3) adjusting parameters and buffer size of *hisat-3n-table* to reduce frequent synchronization; (4) using lock-free queues and batch block processing techniques to reduce frequent locking operations and contention; (5) accelerating loading and output by using OpenMP to parallelize locally independent parts and delaying string assignment; (6) replacing the standard STL memory allocator with *jemalloc* to optimize memory management in multi- threaded environments; (7) partitioning the input set to achieve process-level parallelism; (8) separating dedicated input/output threads and adding input/output buffers in *hisat-3n*.

Experimental results demonstrate that these optimizations collectively reduce the workflow's execution time by **37.2 minutes**, achieving up to a **17.6x speedup** com-

pared to the baseline. This significant improvement in computational efficiency, coupled with high detection accuracy, makes m^5C-UBSseq a more robust solution for large-scale m^5C studies and clinical applications. Our work not only advances the field of RNA modification detection but also provides a scalable and reproducible framework for future bioinformatics research.

KEYWORDS: m^5C **Modification site detection, lock-free queue, Workflow optimization, Bioinformatics tools**

6.2 Introduction

In the field of molecular biology, RNA modifications play a pivotal role in regulating gene expression, post-transcriptional processing, and protein synthesis. Among more than 170 identified RNA modifications, 5-methylcytosine (m^5C) is a critical epigenetic marker that influences the stability, localization, and interaction of RNA with proteins. Despite its biological significance, accurate detection of m^5C sites remains a challenge due to limitations in existing high-throughput sequencing (HTS) methodologies. Techniques such as RNA-Bis-seq and UBS-seq rely on cytosine-to-thymine (CT) conversion signals, which inherently introduce high false-positive rates, compromising the reliability of downstream analyses.

Current workflows for m^5C detection often depend on single-tool approaches, which lack the robustness to balance sensitivity, specificity, and computational efficiency. These limitations hinder large-scale studies and clinical applications where precision and reproducibility are paramount. To address these challenges, the m^5C-UBSseq task integrates a multi-stage bioinformatics pipeline, combining adapter trimming, rRNA/tRNA filtering, genome alignment, and stringent statistical filtering. This workflow aims to minimize false positives while maintaining high detection accuracy and optimizing computational performance.

The core challenge lies in refining the pipeline to enhance both biological validity and technical efficiency. Key steps include constructing CT-converted reference indices, implementing parallelized alignment and deduplication, and applying rigorous statistical thresholds for site validation. By leveraging tools like *hisat-3n*, *samtools*, and custom Python scripts for data consolidation and hypothesis testing, the pipeline ensures robust identification of m^5C candidates across replicates. Furthermore, performance optimizationthrough containerization, parallel processing, and algorithmic improvementsis critical to reducing runtime without compromising accuracy.

This proposal outlines a systematic approach to achieving the dual goals of high precision (95% true positives) and computational efficiency. By integrating modular bioinformatics tools, automating workflow execution, and validating results against

standardized benchmarks, the proposed solution aims to advance m^5C research, enabling scalable and reproducible analyses in both academic and clinical settings.

6.3 Environment

In this section, we provide an overview of the hardware and software components required for running the RNA m^5C modification detection workflow. Each software package is described briefly, followed by detailed explanations of the configuration and build processes for each tool.

6.3.1 Hardware and Software

Table 6.1 and Table 6.2 shows hardware and software configuration.

Item	Model
CPU	Intel® Xeon® Platinum 8358P CPU @ 2.60GHz
RAM	512GB
# of Nodes	3

Table 6.1 Hardware Environment for one CPU node

Item	Model
Operating system	Linux 5.15.0-78-TH-cn x86_64
Compiler	g++ (Ubuntu 11.4.0-1ubuntu1 22.04) 11.4.0

Table 6.2 Software Environment

6.3.2 Installation Procedure

6.3.2.1 Environment Variables

In the following text, we will use `toolroot` to represent the directory path where all tool dependencies are located on our cluster. The *CC* and *CXX* environment variables are set to *gcc* and *g++*, respectively. Additionally, *CXXFLAGS* is set to "-O3 ‘jemalloc-config –libdir‘/libjemalloc.a ‘jemalloc-config –libs‘" to enable the use of the *jemalloc* memory manager. In our integrated workflow package, the environment can be set up with a single command by running the `build.sh` script. Anyone can install and execute the entire workflow by reading the **readme.md** in the integrated workflow package

6.3.2.2 jemalloc Installation

jemalloc is a high-performance, thread-safe memory allocation library, often used to replace standard memory allocators (such as the one in `glibc`). It performs exceptionally well in multithreaded, concurrent environments. The installation command is as follows:

```
cd $toolroot/jemalloc-5.3.0
./configure --prefix=$toolroot/jemalloc-5.3.0/build
make clean
make -j
make install
```

6.3.2.3 hisat-3n Installation

hisat-3n is an efficient genome alignment tool primarily used for aligning high-throughput sequencing data, especially RNA-Seq data. In addition to standard alignment functions, *hisat-3n* can also be used to identify various RNA modification sites. We have prepared the modified *hisat-3n* source code, which only requires recompilation. The following command can be used:

```
cd $toolroot/hisat-3n
rm CMakeCache.txt
cmake .
make clean
make -j
```

6.3.2.4 samtools Installation

samtools is a widely used bioinformatics toolkit primarily designed for processing, analyzing, and manipulating sequencing-related files, particularly those in SAM (Sequence Alignment/Map), BAM (Binary Alignment/Map), and CRAM formats. As we have not modified the *samtools* source code, it can be downloaded from the internet or compiled directly from our packaged source code. The instructions for the latter are as follows:

```
cd $toolroot/\textit{samtools}-1.21
make clean
./configure --prefix=$toolroot/\textit{samtools}-1.21/
   samtoolslocation
make -j
make install
```

6.3.2.5 java and UMICollapse Installation

UMICollapse.jar is a tool designed for deduplicating RNA-seq data by identifying and removing PCR-induced duplicate sequences, thereby improving the accuracy of the data. Java is used to run *UMICollapse.jar*. When using our workflow package, only the environment variables need to be set. The command is as follows:

```
export JAVA_HOME=$toolroot/java/jdk-23.0.1
export PATH=$JAVA_HOME/bin:$PATH
export UMIroot=$toolroot/UMICollapse
```

6.3.2.6 cutseq Installation

cutseq is a tool used for processing RNA sequencing data, primarily for operations such as sequence truncation, trimming, and filtering in high-throughput sequencing data (e.g., FASTQ files), especially in RNA modification-related studies. It is primarily used for preprocessing sequencing data and is managed via Conda. The command is as follows:

```
pip install cutseq
```

6.3.2.7 seqkit Installation

seqkit is an efficient bioinformatics toolkit designed for processing and manipulating biological sequence data, particularly FASTA and FASTQ format genomic or sequencing data. We primarily use it for dataset splitting. When using our workflow package, only the environment variables need to be set. The command is as follows:

```
export PATH=$PATH:$toolroot
```

6.4 Workflow Analysis

The entire workflow can be devided into three parts, including

Part I: Reference Genome Index Construction. This part involves building CT-converted reference indices for rRNA, tRNA, and the genome using *hisat-3n-build* and *samtools*. The generated indices (e.g., FASTA, FAI, SAF) are essential for subsequent alignment and filtering steps.

Part II: Data Processing and Analysis. This stage includes adapter trimming, rRNA/tRNA filtering, genome alignment, deduplication, and site calling using tools like *cutseq*, *hisat-3n*, and *UMICollapse*. The output includes cleaned reads, alignment statistics, and candidate m^5C sites.

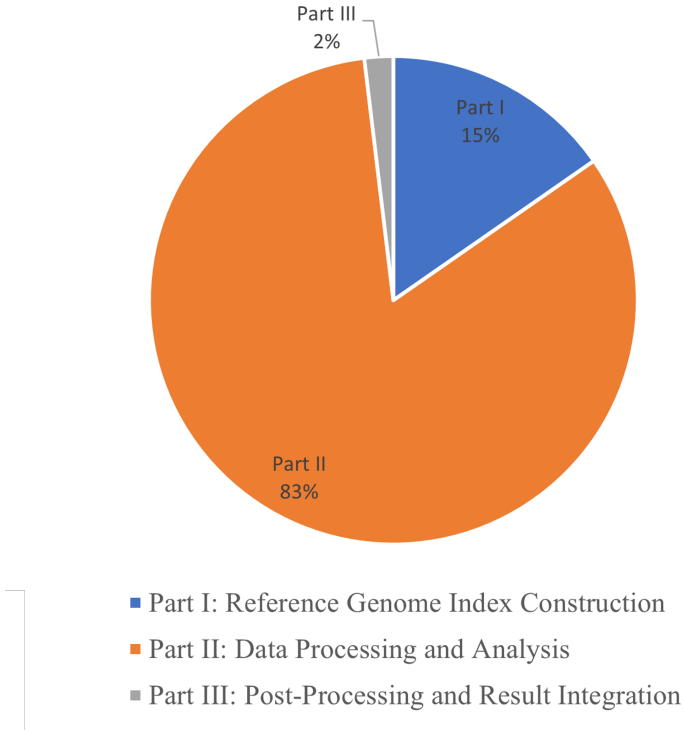

Figure 6.1 Time Consumption Ratio of Each Part

Part III: Post-Processing and Result Integration. In this final phase, candidate sites are filtered and validated through statistical testing (`select_sites.py`, `filter_sites.py`), and results are consolidated into group-level files. The output includes high-confidence m^5C sites and background methylation ratios.

The time distribution across the three components is illustrated in Figure 6.1. Among these, the second component, which consumes the most time, involves several tools, including *hisat-3n-table*, *hisat-3n*, *samtools*, and *UMICollapse*. The time consumption ratio for these tools under the full-core baseline is shown in Figure 6.2. Based on the time consumption distribution, further analysis are conducted on *hisat-3n-table*, *hisat-3n*, *samtools*.

6.4.1 hisat-3n-table Analysis

hisat-3n-table is primarily used to generate alignment result statistics. It analyzes alignment results (SAM/BAM files) and produces detailed statistical tables that provide data on alignment quality, alignment rate, coverage, error rate, and more. The tool consists of a main thread, worker threads, and an output thread. The main thread is responsible for loading the reference genome data and the aligned data, then transferring the processed data to the output buffer. The worker threads handle the parsing and processing of the input data, while the output thread writes the data from the output buffer to disk.

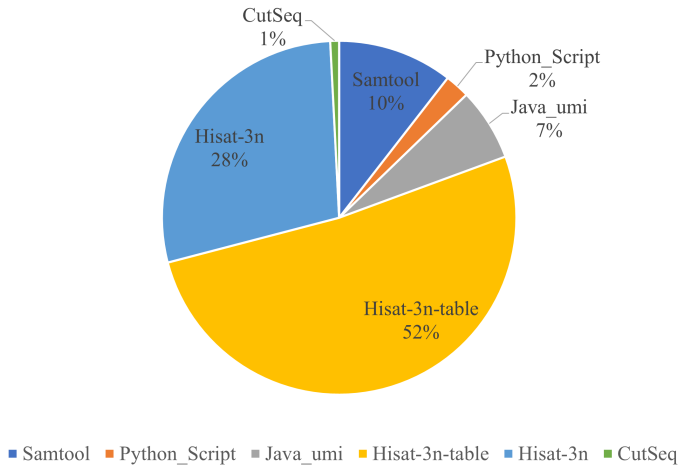

Figure 6.2 Time Consumption Ratio of Each Tool in Part II and III

hisat-3n-table's simplified logic is shown in Figure 6.3. Based on our experiments and analysis, *hisat-3n-table* exhibits poor scalability in multi-threading. When the number of threads is increased beyond a certain threshold, significant time is consumed by lock contention and synchronization operations between the main thread, worker threads, and output threads. Particularly under multi-threaded concurrency, the main thread becomes a performance bottleneck, with most worker threads waiting for the main thread to provide data.

6.4.2 hisat-3n Analysis

The alignment process of `hisat-3n` consists of the following key steps:

1. **Loading the Genome Index**: The alignment process requires the use of a pre-built genome index, which is constructed using *hisat-3n-build*.

2. **Reading Input Sequencing Data**: RNA-Seq data is typically stored in FASTQ format. When a worker thread finishes processing the current data, it requests the next data set, which can either be single-end or paired-end. For single-end data, the thread reads one read from the FASTQ file at a time, extracts the data from the FASTQ format, and processes it. During the reading process, locks are used to maintain atomicity of operations.

3. **Aligning the Read Sequences to the Reference Genome**: The RNA-Seq sequencing data is efficiently aligned to the reference genome using algorithms based on BWT (Burrows-Wheeler Transform), FM-index, and graphical alignment models.

4. **Outputting the Alignment Results**: Once a worker thread completes the alignment of a read, it calls the output interface to write the processed results to the output buffer and updates the relevant parameters. Each output operation is

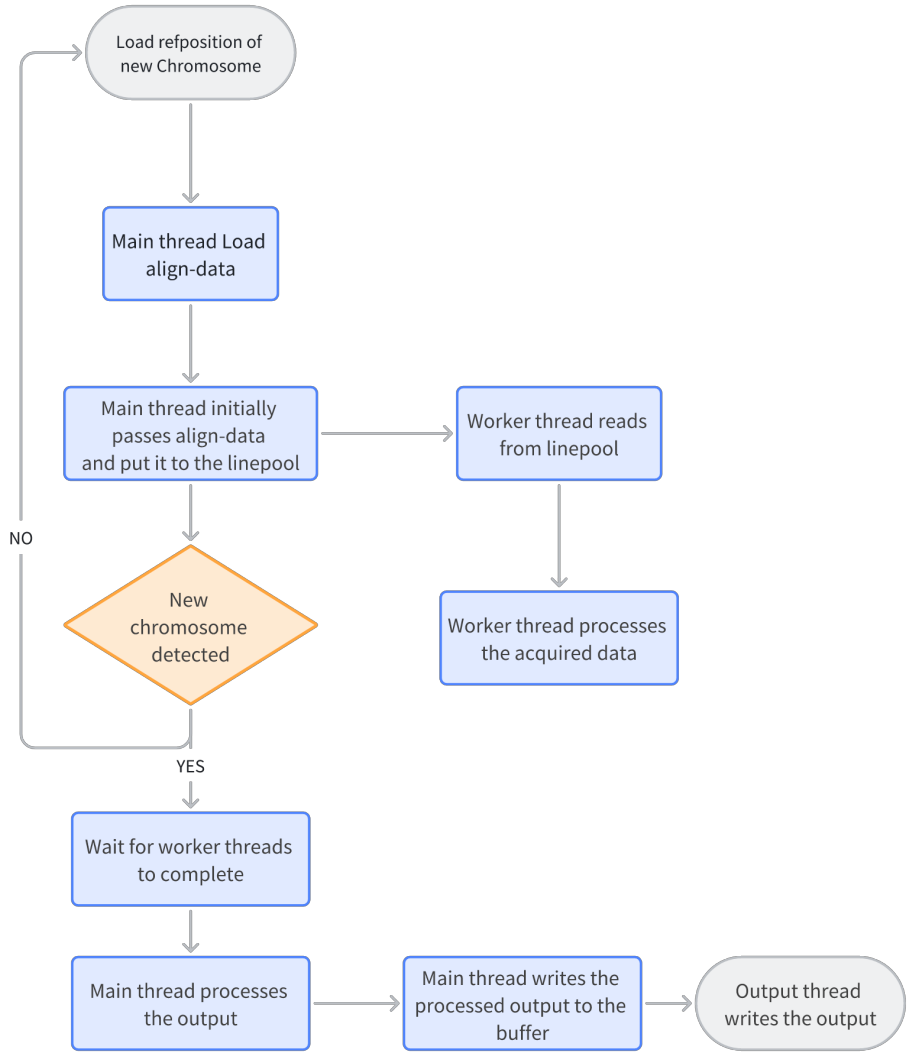

Figure 6.3 Software Analysis of hisat-3n-table

protected by a lock to ensure atomicity.

Based on our experiments and analysis, the thread scalability of the *hisat-3n* alignment operation is primarily limited by input and output processes. Specifically, access to critical resources during input and output leads to significant lock contention and busy-waiting, which negatively impacts performance.

6.5 Optimization

6.5.1 Setting up Baseline

In this work, we conducted our baseline experiments on one compute node with all 64 cores. The execution time for each part of the workflow is summarized in Table 6.3.

Test Case	Baseline
Whole Workflow	651.6min
Part II(on single dataset)	221.min(avg)
Part II(total)	645min

Table 6.3 m^5c USB-seq Baseline

6.5.2 Optimization of the Entire Workflow

6.5.2.1 Parallelization of the Workflow Using Bash

Upon analyzing the workflows contextual dependencies, we identified several steps that could be parallelized to utilize available computational resources better. The parallelized workflow is depicted in Figure 6.4.

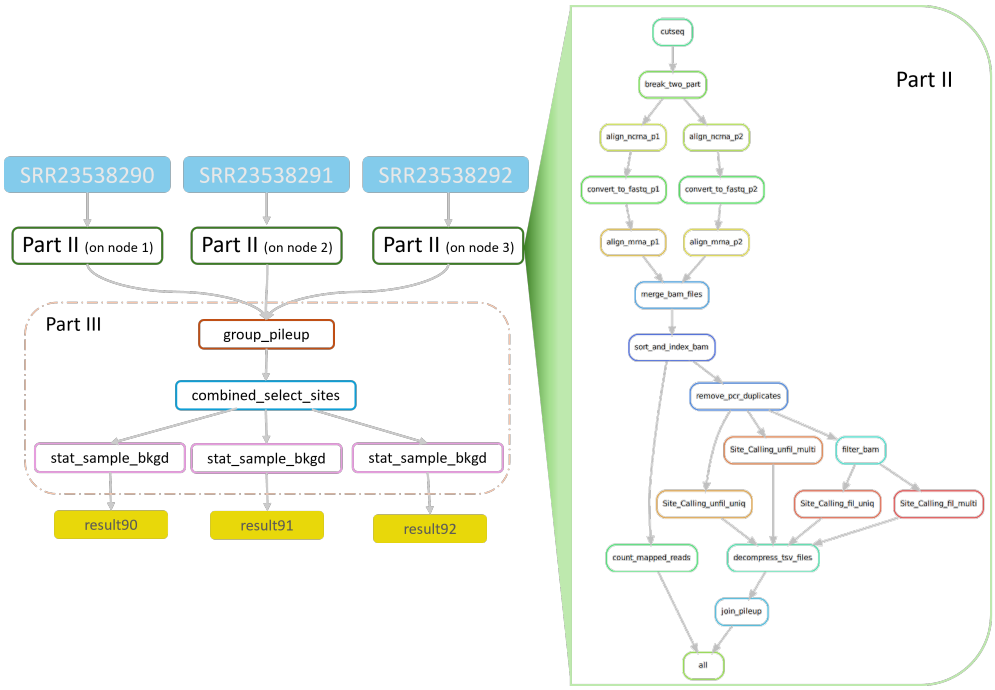

Figure 6.4 Parallelized Workflow

The processing of the three datasets in Part II can be distributed across three nodes for parallel execution. Additionally, as illustrated in the workflow, for each dataset, the data cleaned by *cutseq* can be split into smaller subsets, which allows for parallel execution of the following steps without interdependencies. This parallelization significantly reduces the processing time of time-consuming operations like mapping and conversion.

In particular, before the sorting and deduplication steps, the data is merged for processing. Steps that have minimal logical dependencies on each other, such as mapping with *hisat-3n* and filtering with *samtools*, can run concurrently. By parallelizing these

steps, we can fully exploit the available cores and improve overall throughput.

For instance, during the alignment phase, while one part of the data is being mapped, another can undergo filtering or further processing, all in parallel. The optimization effectively reduces the overall execution time by distributing the workload across multiple cores.

Furthermore, task-level parallelism was implemented in Part III (Post-Processing and Result Integration) to optimize computationally intensive steps such as `join_pileup` and `stat_sample_background`. These operations, which involve statistical aggregation and background methylation analysis, were executed concurrently across independent datasets (ID90, ID91, and ID92) to eliminate sequential dependencies. As illustrated in Figure 6.4, each dataset was processed in parallel by distributing tasks across available cores, thereby minimizing idle resources and reducing synchronization overhead. By implementing this strategy, the execution time for Part III was reduced by **38%**, achieving a **1.6x speedup** compared to the baseline sequential execution.

This parallelization strategy is applied throughout the workflow, where independent or logically separable steps are executed simultaneously. This not only accelerates the overall process but also ensures better resource utilization across multiple processing units.

6.5.2.2 Run in Memory Filesystem

Considering that the entire workflow involves frequent reading and writing of large files, we optimized the process by loading all required files into memory filesystem. This approach effectively mitigates the need for repeated disk I/O operations, which are typically a major source of latency. By storing files in memory rather than on non-volatile storage media, we significantly reduced I/O delay, leading to faster execution times and improved overall performance. This optimization was particularly effective in steps involving *samtools*, where we observed an impressive speedup of up to **6x**. A comparison of the execution times for various *samtools* commands between the in-memory filesystem and the disk-based storage is provided in Figure 6.5

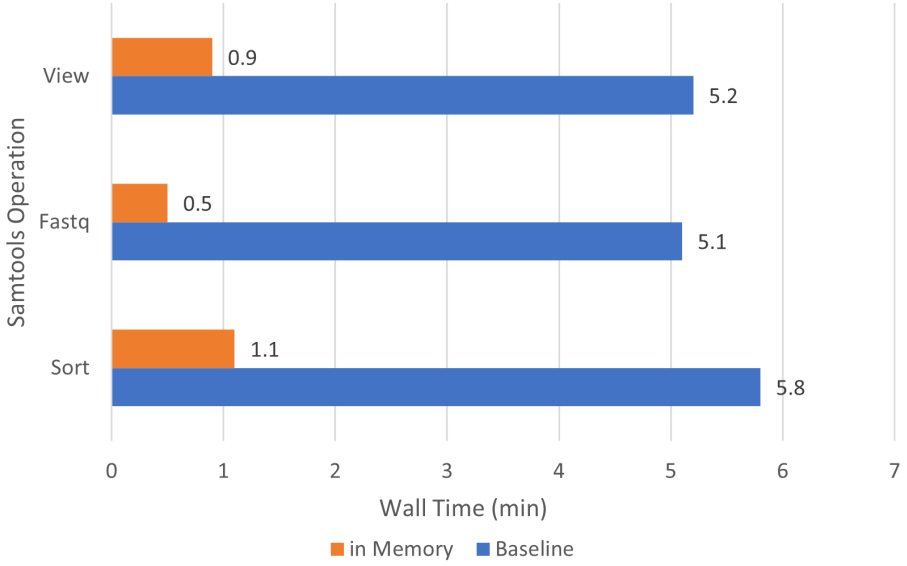

Figure 6.5 Execution Time Comparison of *samtools* Commands: In-Memory Filesystem vs. Disk-Based Filesystem

6.5.3 Optimization of hisat-3n-table

hisat-3n-table is an auxiliary tool built into *hisat-3n*, primarily used in Part II's *Site calling and filtering* phase, and it accounts for a significant portion of the time spent in both Part II and the entire workflow.

In this section, we begin by providing an overview of the application logic of *hisat-3n-table* and establish a baseline. We then proceed to optimize the application based on its characteristics and performance bottlenecks. To address issues related to suboptimal thread scalability and contention for critical resources, we introduced lock-free queues to safeguard critical sections and employed batch processing techniques to minimize resource contention.

To optimize hotspots in both the main and worker threads, we implemented strategies such as deferred assignments, parallel execution of independent code sections, and function call optimizations to accelerate thread processing. Given the applications frequent memory operations, we replaced the standard STL memory allocator with *jemalloc*, which is better suited for multi-threaded, high-concurrency, and high-performance environments.

Lastly, to further enhance the scalability of the application, we proposed a novel chromosome-level parallelization strategy, which reduces the overhead in the main thread for each process, thereby maximizing overall efficiency. As a result, we achieved an acceleration factor of nearly **17x** for this application.

6.5.3.1 Baseline of hisat-3n-table

To establish a performance baseline, we first configures the compiler, compilation options, and thread settings for the application.

Compiler and Compilation Options We use **g++ 11.4** as the compiler, with the following optimization settings:

- **-O3**: Aggressive optimizations for speed.
- **-funroll-loops**: Loop unrolling to improve performance.
- **-march=native**: Architecture-specific optimizations.
- **-fopenmp**: Enable OpenMP for parallelism.
- **-flto=4**: Link-Time Optimization for better performance.
- **-faligned-new**: Align memory allocations for efficiency.
- **jemalloc**: Replacing the default allocator with jemalloc for better memory management in a multi-threaded environment.

The full compilation command is as follows:

```
g++ -O3 -funroll-loops -march=native -fopenmp -flto=4 -
   faligned-new -L/GLOBALFS/sysu_hpcscc_2/lh/asc25/depend/
   jemalloc-5.3.0/build/lib -Wl,-rpath,/GLOBALFS/
   sysu_hpcscc_2/lh/asc25/depend/jemalloc-5.3.0/build/lib -
   ljemalloc -lm -lstdc++ -pthread
```

Thread Configuration We set **60 worker threads** (`-p 60`), considering the main thread, an output thread, and 60 worker threads to handle parallel computation efficiently across the system's cores.

Baseline Execution To establish the baseline, we run the following commands:

```
hisat-3n-table -p 60 -u --alignments $dataroot_shm/temp.sam
   --ref $dataroot_shm/Homo_sapiens.GRCh38.dna.
   primary_assembly.fa --output-name \$dataroot_shm/
   temp_output.tsv --base-change C,T
```

This baseline configuration resultes in a runtime of **21.9 minutes**. This serves as the reference for evaluating the effectiveness of subsequent optimizations.

6.5.3.2 Adjust Parameters and Buffer Size

`hisat-3n-table` involves a significant number of `sleep` operations. Specifically, when the buffer size is too large, some threads are forced to suspend, waiting for other threads to consume the buffer to prevent it from becoming excessively large. Although the operations performed by these threads are independent, the small buffer size leads to frequent synchronization and waiting, which results in underutilization of computational resources. To address this, we optimize the waiting conditions for the `sleep` operation and adjust the buffer's critical size. This improves thread activity and parallelism, maximizing the computational power of multi-core processors.

Additionally, when `samPos` exceeds `reloadPos`, the main thread frequently invokes operations like `positions->appendingFinished()`, `moveBlockToOutput()`, and `loadMore()`. Specifically, `appendingFinished()` forces synchronization between the main thread and worker threads, causing the main thread to wait for the worker threads to complete all tasks in the current buffer. Meanwhile, the `moveBlockToOutput()` and `loadMore()` operations introduce significant contention. To mitigate this, we adjust the `loadingBlockSize` parameter, increasing the buffer size to allow more `refposition` data to be loaded at once. This reduces synchronization frequency and contention, improving data loading efficiency.

Through these optimizations in buffer size and related parameters, we observe significant performance improvements in practical applications. The improvement in performance is as shown in the Figure 6.6.

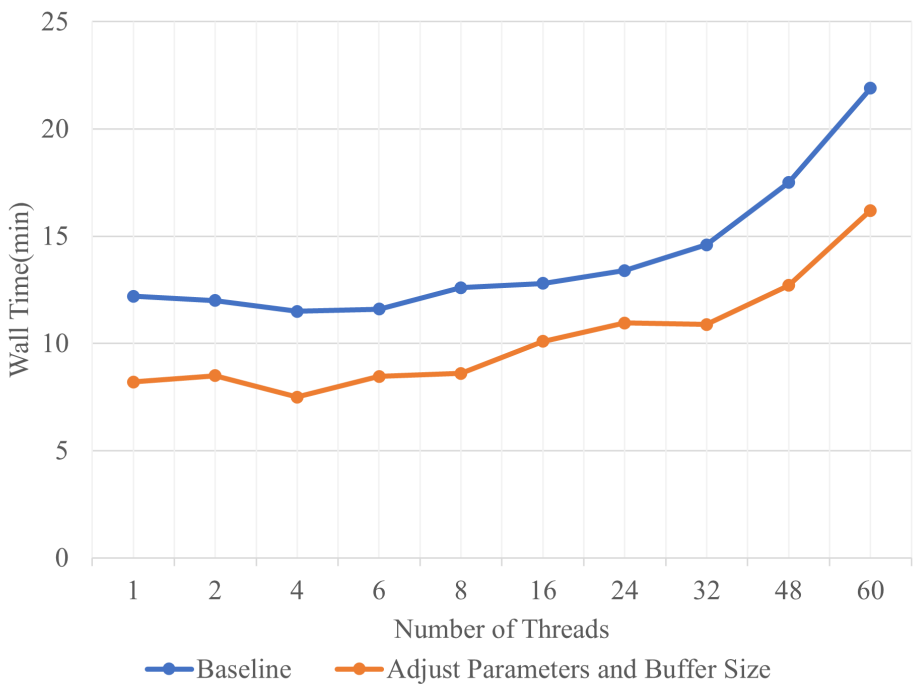

Figure 6.6 Speedup of Adjust Parameters and Buffer Size

6.5.3.3 Use Lock-free Queue

During the performance analysis of the application, we discovered that lock and unlock operations consumed a significant amount of system resources. The original algorithm involved multiple buffers (e.g., `outputpool`, `linepool`, `freepositionpool`), each of which was a unique and globally shared critical resource implemented through thread-safe queues. When different threads attempted to access the same buffer, they had to perform lock and unlock operations. While this locking mechanism ensures data consistency, it also introduces considerable performance overhead. Specifically, the main areas of contention included:

- Contention between the main thread and worker threads (`linepool`)
  - The main thread reads each line of data from the alignment file into the `linepool`.
  - Worker threads extract data lines from the `linepool` for processing.
- Contention between the main thread and the output thread (`outputpool`)
  - The main thread places processed `refposition` data into the `outputpool`.
  - The output thread retrieves processed results from the `outputpool` for output.
- Contention among worker threads
  - Multiple worker threads attempt to access the same `position` (this will be discussed in detail later).
  - Multiple worker threads simultaneously access the `linepool`.

Due to the frequent lock and unlock operations performed by threads accessing these critical resources, significant performance overhead was incurred. At the same time, the contention led to a large number of threads being put into a sleeping state, wasting valuable computational resources. This explained why the execution time of the application increased as the number of threads grew—more threads introduced more contention and busy-waiting.

To address this issue, we replaced the original `safequeue` implementation with a more efficient and safe lock-free queue. For buffers with no potential contention (such as `freepositionpool`), we used a standard queue as a replacement.

Specifically, we replaced the `outputpool`, `linepool`, and other buffers with lock-free queues, utilizing moodycamel::ConcurrentQueue[1] to enhance runtime efficiency. The moodycamel library is an efficient concurrent data structure library in C++, and the `ConcurrentQueue` is designed for high-performance data transfer between multiple

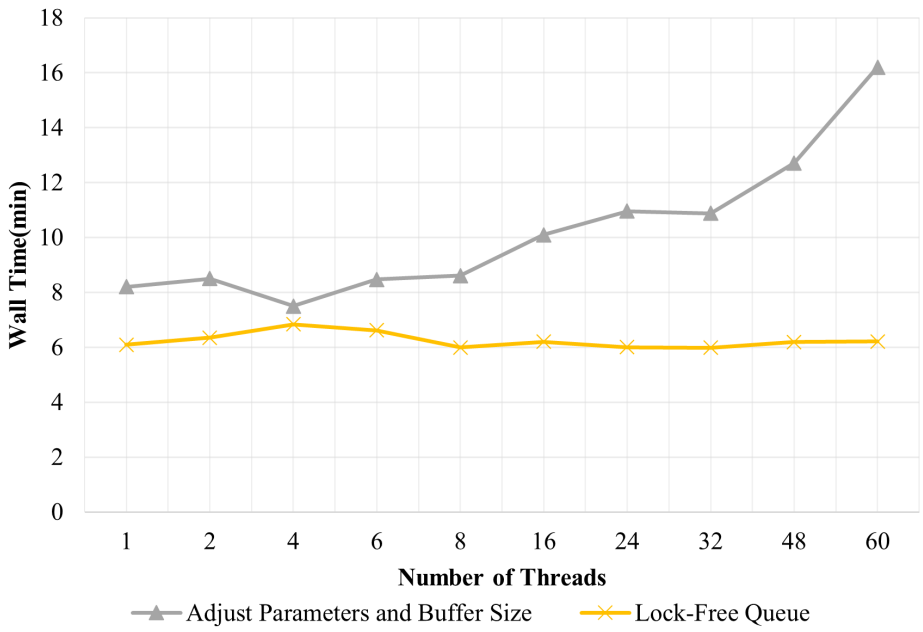

Figure 6.7 Speed Up of Lock-Free Queue across Different Number of Threads

producer and consumer threads, reducing performance bottlenecks caused by lock contention.

However, the moodycamel library does not provide a precise API to describe the current size of the queue (it only offers the imprecise `size_approx` method). Therefore, to achieve precise control flow, we declared an atomic variable for each critical resource. When data is enqueued, the atomic variable is incremented; when data is dequeued, the atomic variable is decremented. In this way, the main thread can monitor these atomic variables to accurately determine if any queue is empty, thus enabling more efficient resource management and control. In practice, the atomic increment and decrement operations usually only require atomicity without memory synchronization, so the overhead of adding an atomic variable to monitor buffer size is minimal.

For `freepositionpool`, we found that this buffer is only produced and consumed by the main thread, and both actions do not occur simultaneously. Therefore, it is safe to use a standard queue instead of a lock-free queue, further reducing unnecessary complexity.

Through these optimizations, combined with the earlier adjustments to buffer sizes, we significantly reduced the overhead of lock contention and synchronization operations, improved the efficiency of multithreaded processing, and minimized performance wastage caused by unnecessary thread sleeping and busy-waiting. The speedup achieved by using a lock-free queue under different thread counts is illustrated in the Figure 6.7.

6.5.3.4 Batch Block Processing

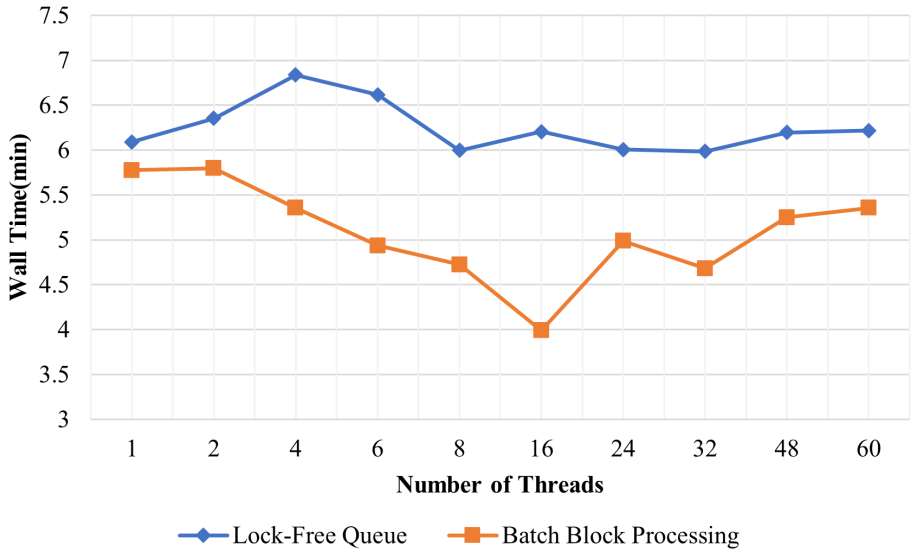

Figure 6.8 Speedup of Batch Block Processing

In the original implementation, multiple worker threads encountered lock and contention issues when accessing the same `Position`, particularly when processing sorted input files. Adjacent alignment lines were likely to map to the same `Position`, which increased the probability of thread conflicts, leading to performance bottlenecks and thread scheduling overhead.

To mitigate the lock and contention issues caused by concurrent access, we adopted a batch processing approach. In this solution, worker threads no longer process each line individually. Instead, they read multiple lines from the line buffer, `linepool`, at once, treating these lines as a block for processing. The thread only begins processing the block once it reaches a certain size, thereby preventing multiple threads from simultaneously handling adjacent lines and reducing lock contention and competition. The acceleration performance of block processing under different thread counts is illustrated in the Figure 6.8. The modified code of lock-free queue and batch block processing is shown in appendix 6.6.

6.5.3.5 Iterative searchReadNameID

In the original implementation, the `searchReadNameID` method used a recursive binary search algorithm to locate the target `readID` in a sorted array. Although the binary search algorithm itself has a low time complexity $O(\log n)$, the recursive approach consumed a significant amount of worker thread resources in large-scale data processing, leading to a performance bottleneck.

We replaced the recursive binary search with an iterative version to reduce the over-

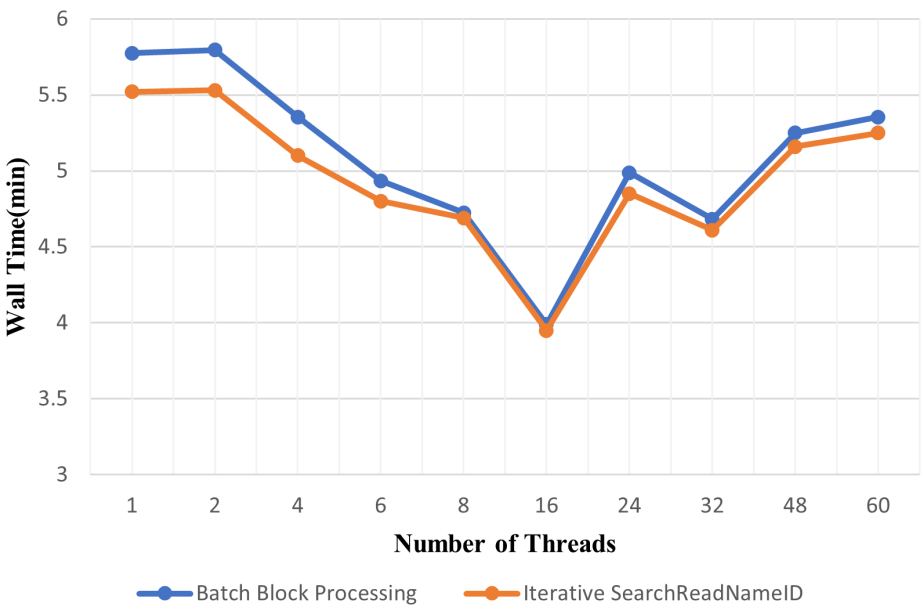

Figure 6.9 Speedup of Iterative SearchReadNameID

head of function calls. Recursive calls consume stack space and involve multiple function entry and exit operations, which significantly deplete worker thread resources. By adopting an iterative approach, we eliminated the additional overhead of recursive calls, reducing stack space usage and function call frequency, and thereby improving overall performance. The speedup achieved by using iterative searchReadNameID under different thread counts is illustrated in the Figure 6.9.

6.5.3.6 Faster Loading and Output

After effectively eliminating lock contention, the next performance bottleneck shifted to operations within the main thread. The main thread primarily handles two critical tasks: loading information (including loading reference genome data and the alignment file) and output operations (writing the processed results to the output buffer). Specifically, the execution times of the `loadNewChromosome` and `moveAllToOutput` functions were relatively long, becoming key performance bottlenecks.

- For `loadNewChromosome`, we postponed redundant string assignment operations to more suitable moments, avoiding unnecessary overhead in the main thread.
- For `moveAllToOutput`, we parallelized the independent and parallelizable parts of the operation, improving the efficiency of the output process.

This optimization effectively reduced the workload of the main thread and further enhanced overall performance, which can be seen in Figure 6.16.

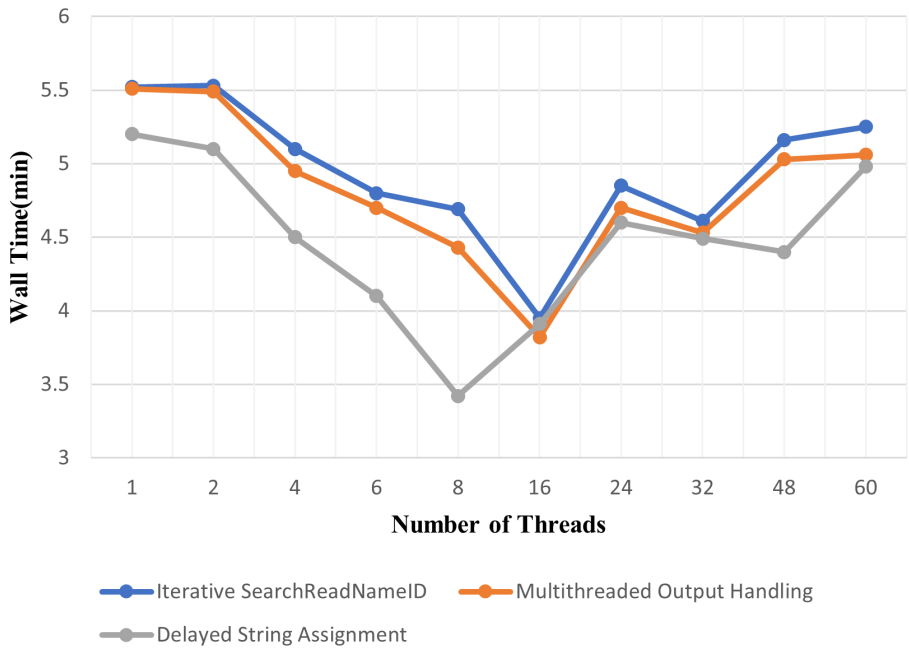

Figure 6.10 Speedup of IO Optimization

Delayed string assignment We implemented delayed string assignment using the `set_without_string` function. In the original implementation, every time a new chromosome was read (i.e., in the `loadNewChromosome` function), the main thread would assign the corresponding chromosome information (`chromosome`) to each read `Position` object. This led to a significant number of redundant memory assignment operations, especially involving deep copies of strings (`string = copy`), thus wasting valuable computational resources.

In reality, the `chromosome` information of a `Position` is only used when the data passes validity checks and is ready for output. Moreover, only the `Position` objects that pass the validity check are output. Since the number of `Position` objects that pass the validity check is much smaller than the total number of reference positions (`refPositions`), postponing the assignment of the `chromosome` string until the output stage effectively reduces unnecessary memory assignments, significantly improving the execution efficiency of the main thread.

Multithreaded output handling In the `moveAllToOutput` function, the main thread needs to iterate over each element in the `refPositions` container and perform a validity check. For valid elements, the main thread outputs them to the output buffer; for invalid elements, it performs resource deallocation operations.

The validity check and resource deallocation operations in this function are independent, meaning they can be executed in parallel. We employed OpenMP to parallelize these two tasks. Specifically, multiple threads concurrently check the validity of each

element, releasing resources when an element is deemed invalid and saving the element when it is valid. It is important to note that each thread independently performs the resource deallocation, which is highly compatible with the parallel optimization of the `jemalloc` memory allocator, ensuring the efficiency of the memory release process.

Finally, the main thread is responsible for batch outputting all valid elements to the output buffer. By parallelizing the validity checks and resource deallocation operations, we significantly improved the execution efficiency of the `moveAllToOutput` function and reduced the workload of the main thread.

6.5.3.7 Memory Management Optimization

In the original implementation, we observed that frequent memory management operations (such as `malloc` and `free`) had a significant impact on performance. To improve the efficiency of memory operations, we introduced the **jemalloc** memory manager and leveraged the system's native memory reclamation mechanism, replacing manual `delete` operations at the end of the program. This change accelerated the memory management process.

Jemalloc Memory Allocation Library In the original implementation, we utilized resource pools (such as `freepositionpool` and `linepool`) to reduce the frequency of `new` and `delete` operations, thereby avoiding redundant memory allocations and deallocations. However, the initialization of the resource pools and their reallocation when the pool capacity was insufficient still caused considerable performance overhead, particularly under high-load conditions.

Additionally, the traditional STL `malloc` implementation has certain efficiency bottlenecks, especially in multi-threaded environments. Frequent memory allocation and deallocation operations in such environments can lead to severe contention, resulting in poor concurrency performance on multi-core processors. Since the standard memory allocator is not optimized for multi-threading, its efficiency in high-concurrency scenarios is inadequate.

To address these issues, we introduced **jemalloc** as a new memory allocation scheme. jemalloc is a high-performance memory management library specifically designed for multi-threaded environments. By employing refined memory pool management and concurrency optimizations, jemalloc significantly enhances the performance of memory allocation and deallocation. Particularly on multi-core systems, jemalloc reduces contention during memory allocation, substantially improving overall performance. Testing showed that using jemalloc resulted in a performance improvement of approximately 10%.

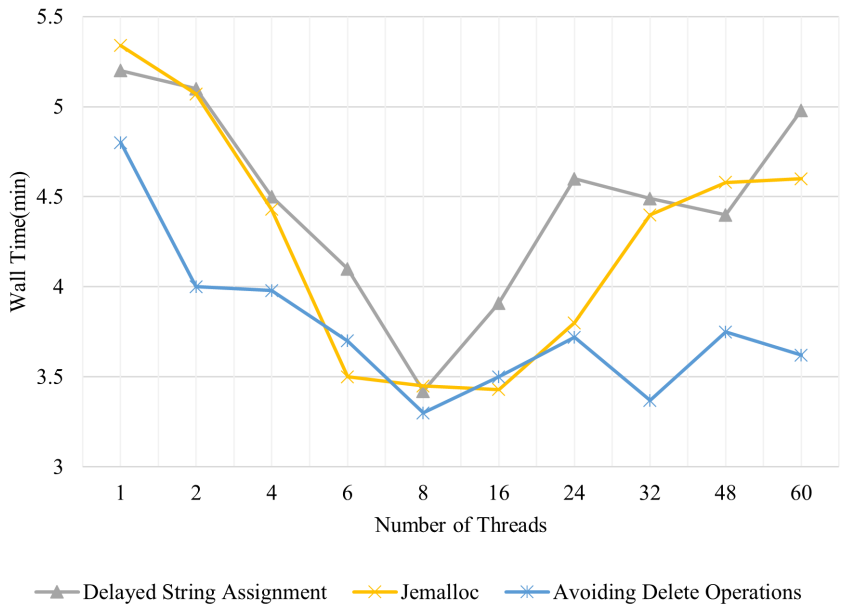

Figure 6.11 Performance Improvement Achieved by Optimizing Memory Operations

Avoiding `delete` Operations at Program Exit During the program's termination phase, we noticed a large number of redundant `delete` operations. Since the operating system automatically reclaims all allocated memory when the program exits, these `delete` operations were unnecessary and introduced avoidable performance overhead. To optimize execution efficiency, we chose to skip these redundant memory deallocation operations, relying instead on the operating system's memory reclamation mechanism. After removing these operations, the program's execution speed improved by approximately 10%.

The specific speedup results are displayed in the Figure 6.11.

6.5.3.8 Process Parallelism with Chromosome

After the aforementioned optimizations, the scalability of the tool in terms of threading was further limited by the main thread's loading and output operations. To alleviate the pressure on the main thread, we implemented process-level parallelism based on different input chromosome sets. When the number of threads is large, multiple processes utilizing the same number of threads as a single process result in far better performance through chromosome-level parallelism.

Our observations indicated that each chromosome's data is the largest independent unit and thus suitable for parallel processing. To better utilize computational resources, we proposed an optimization strategy where the chromosome set is partitioned and processed in parallel. By launching multiple processes to handle different chromosome subsets, the main thread's burden is effectively reduced, and overall pro-

gram parallelism is improved. The specific approach is as follows:

Chromosome Set Partitioning: All chromosome data is divided into multiple subsets based on a set rule, where each subset corresponds to an independent processing unit.

Parallel Processing: Multiple `hisat-3n-table` processes are launched, with each process handling a different chromosome set to achieve parallel computation across the sets.

Result Merging: The results generated by each process are finally merged to ensure consistency with the output from a single-process execution.

In practice, I implemented a C++ program that automatically divides the chromosome sets and, based on the partition points, uses `sed` to generate different input sets. Subsequently, I launched N processes, with each process handling a different chromosome set. Finally, I used `cat` to merge the results. All these operations were integrated into a bash script, and the following command was executed:

```
./hisat-3n-table-bingxing.sh \$progress_num $filename
  $thread_num
```

In earlier optimizations, we found that the optimal time was achieved when a single process launched 6–16 worker threads. Here, we start 6 processes, each with 8 threads, resulting in approximately 60 threads when including the main and output threads. We compared the effect of this setup with the baseline and previously optimized results, as shown in Figure 6.12.

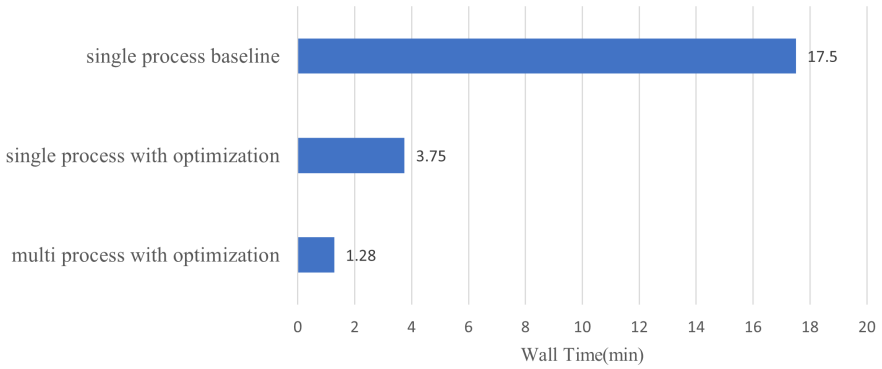

Figure 6.12 Single Process vs Multiple Processes with Approximately the Same Total Number of Threads

6.5.3.9 Final Result

To test the optimization effects of `hisat-3n-table`, we performed the following experiments:

```
1    hisat-3n-table -p $thread -u --alignments temp.sam --ref
        $path_to_GRCh38/Homo_sapiens.GRCh38.dna.
        primary_assembly.fa --output-name temp_output.tsv --
        base-change C,T
```

In the single-process implementation, we found that performance was optimal when using 6–8 worker threads. By adjusting the buffer size and related parameters, adopting lock-free queues, implementing batch block processing, delaying string assignments, and introducing the jemalloc memory manager, the acceleration effect for the single-process implementation was significant. The overall speedup of hisat-3n-table is illustrated in Figure 6.13.

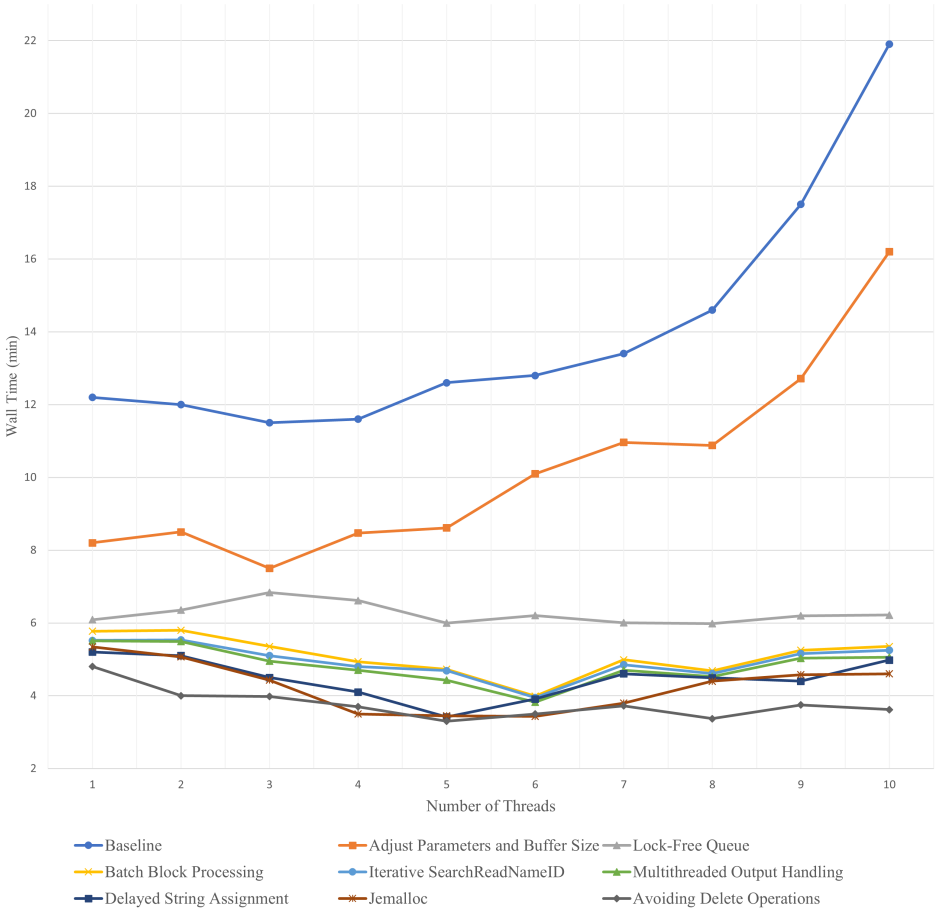

Figure 6.13 Overall SpeedUp of Hisat-3n-table on Single Process

In the multi-process implementation, to make the best use of all computing cores, we employed 4 processes, with each process running 6–8 threads. This configuration further improved performance. As shown in Figure 6.12.

Overall, the original implementation of `hisat-3n-table` was limited by thread-locking mechanisms and competition between threads, which resulted in poor scalability. To

optimize the application, we first focused on resolving issues related to thread synchronization and resource contention. By adopting lock-free queues and batch block processing techniques, we significantly improved concurrent performance.

Secondly, we optimized several time-consuming functions in both the worker and main threads. Specifically, we fine-tuned the `searchReadNameID` function in the worker threads and the input/output operations in the main thread, effectively accelerating the overall runtime.

Furthermore, given the frequent `malloc` and `free` operations in the application, we optimized memory management. By introducing the *jemalloc* memory allocator and removing redundant `delete` operations, we achieved significant improvements in memory usage efficiency and access speed.

Finally, to further enhance the scalability of the application, we implemented a process-level parallelization scheme based on the independence of chromosome sets. This improvement notably boosted performance in multi-core environments.

As a result of these optimizations, we achieved an impressive **17x speedup** in the *hisat-3n-table* tool.

6.5.4 Optimization of hisat-3n

hisat-3n is the primary time-consuming tool in Part II for the **rRNA/tRNA filtering and genome alignment** step. It is a genome alignment tool designed to efficiently align short sequences (such as reads from high-throughput sequencing) to a reference genome. The tool integrates traditional genome alignment methods with modern efficient data structures (e.g., BWT and FM indices) to tackle large-scale and highly complex genome alignment tasks.

In this section, we first analyzed the structure of the application. Given its characteristic of slow input/output operations, we added a dedicated input thread and a dedicated output thread. This setup prevents competition between worker threads for input/output operations. We also employed lock-free queues as buffers between the input/output threads and worker threads, significantly enhancing the application's thread scalability. Furthermore, we observed that the input set has a certain level of independence. By partitioning the input and output sets, we implemented process-level parallelism. As a result, we achieved approximately **13.4x speedup** for this application.

6.5.4.1 Baseline of hisat-3n

To establish a reproducible performance baseline for *hisat-3n*, we first defined the compilation environment, optimization flags, and runtime configurations prior to code

modifications. The baseline configuration was designed to maximize computational efficiency while maintaining compatibility with existing workflow dependencies.

Compiler and Optimization Flags

The tool was compiled using **g++ 11.4** with aggressive optimization flags tailored for modern x86 architectures:

- **-O3**: Enables high-level compiler optimizations for speed.
- **-funroll-loops**: Reduces loop overhead through loop unrolling.
- **-march=native**: Generates architecture-specific optimizations for the target CPU (Intelő Xeonő Platinum 8358P).
- **-fopenmp**: Enables OpenMP-based parallelism.
- **-flto=4**: Performs link-time optimization across translation units.
- **-faligned-new**: Ensures alignment for dynamically allocated memory.
- **jemalloc Integration**: Linked against **jemalloc 5.3.0** (`-L` and `-Wl,-rpath` flags) to optimize memory management in multi-threaded contexts.

Thread Configuration

To fully utilize the 64-core compute node, we allocated **62 worker threads** (`-p 62`), reserving two cores for system processes and I/O operations. This configuration ensured minimal resource contention while maximizing parallel throughput during alignment tasks.

Baseline Execution

The baseline performance was measured using the following command:

```
hisat-3n --index \$dataroot\_shm/Homo\_sapiens.GRCh38.ncrna.
  fa \textbackslash{} --summary-file \$dataroot\_shm/
  map2ncrna.biaozhun.output.summary \textbackslash{} --new-
  summary -q -U \$dataroot\_shm/\$id\_num.fastq\_cut \
  textbackslash{} -p 62 --base-change C,T --mp 8,2 \
  textbackslash{} --no-spliced-alignment --directional-
  mapping \textbackslash{} -S \$dataroot\_shm/temp.biaozhun.
  sam Key parameters include:
```

The baseline execution completed in **20.3 minutes**, establishing a reference for evaluating subsequent optimizations.

6.5.4.2 Run in Memory-file-system

Considering that the output `temp.sam` file is relatively large, disk I/O operations were found to consume a significant amount of time. To mitigate this, we moved the file output operations to an in-memory disk. By performing file operations in memory, we were able to significantly reduce disk I/O latency, thereby enhancing the overall data processing efficiency.

6.5.4.3 Lock-free output

In the original implementation, the `OutputQueue` is a globally shared resource responsible for coordinating interactions between the worker threads and the output buffer. Whenever a worker thread finishes processing a data unit and is ready to output the result, it calls the `OutputQueue::beginRead` and `OutputQueue::finishRead` functions, utilizing locks to ensure thread safety.

Specifically, `beginRead` is mainly used to modify variables for tracking and controlling the output progress, such as incrementing the variable `nstarted_`. On the other hand, `finishRead` not only updates variables (like `nfinished_` and `nflushed_`) but also handles transferring the string `ref` to the output buffer.

To reduce the frequency of lock operations in `OutputQueue`, we implemented the following optimizations:

1. **Narrowing the scope and frequency of locking in `beginRead`:** We replaced the increment of the variable `nstarted_` in `beginRead` with an atomic variable, and used `nstarted_2.fetch_add(1, std::memory_order_relaxed)` for the increment operation. Since this variable does not require frequent synchronization, and `beginRead` primarily performs this operation, this optimization reduces the performance loss caused by locking while maintaining atomicity.

2. **Narrowing the scope and frequency of locking in `finishRead`:** Similarly, we changed the increment operations in `finishRead` (such as `nfinished_++` and `nflushed_++`) to atomic operations, avoiding the performance bottleneck caused by lock contention.

3. **Introducing a lock-free queue `output_queue_2`:** Instead of directly writing to the output buffer, we first cache the worker thread's output results in the lock-free queue `output_queue_2`. This effectively avoids lock contention between worker threads.

4. **Introducing a dedicated output thread:** We added a separate output thread responsible for transferring the data from the lock-free queue `output_queue_2` to the output buffer. This design decouples the worker threads from the output

process, thus reducing synchronization overhead.

5. **Ensuring output completion by the output thread:** At the end of the main thread, we ensure that the output thread has completed all data output. The queueing and dequeuing mechanism, implemented with atomic operations, guarantees that all results are properly outputted, thus avoiding output omissions.

6.5.4.4 Lock-free input

After optimizing the `OutputQueue`, the new parallel bottleneck emerged during the data reading phase, specifically in the `ReadNext` section. In the original implementation, data reading was handled through a single, shared resource class. Each time data was read, threads had to call the `read` method of this class, using locks to ensure thread safety and prevent repeated data reads.

Following the same approach as the optimization for `OutputQueue`, we implemented the following improvements:

- **Introducing a dedicated read thread:** We added a dedicated read thread that continuously performs the data reading operations. By decoupling data reading from the other processing threads, we reduced synchronization pressure.
- **Using a lock-free queue to store read data:** A lock-free queue was created to store all the data read by the dedicated read thread. This avoids thread contention caused by locking during the reading process.
- **Other threads reading data from the lock-free queue:** All other worker threads can now directly retrieve data from the lock-free queue, thereby eliminating the synchronization bottleneck during the reading phase.

To further improve the reading speed and avoid the performance overhead associated with using queues as buffers, we transmit only pointers rather than entire objects in the queues. Additionally, employing a batch processing method for handling data in blocks also contributes to enhancing the reading speed. The modified code is shown in Appendix 6.6.

6.5.4.5 Process parallelism

We found that *hisat-3n* allows for parallel alignment of different reads. Therefore, we can enhance overall performance by launching multiple processes to concurrently process different read sets. The specific implementation steps are as follows:

- Use the `seqkit split2` tool to divide the original `fastq` file into smaller datasets for parallel processing.
- Launch multiple processes to run *hisat-3n* in parallel, with each process handling

an independent input dataset, thereby fully leveraging the computational power of multi-core processors.

- Use the `samtools cat` tool to merge the `.bam` files generated by each process, resulting in the final alignment output.

6.5.4.6 Final result

To test the runtime of the tool, we executed the following commands:

```
hisat-3n --index \$path_to_GRCh38/Homo_sapiens.GRCh38.
   ncrna.fa --summary-file $dataroot/$id_num/map2ncrna.
   output.summary --new-summary -q -U $dataroot/$id_num/\
   $id_num.fastq_cut -p \$thread --base-change C,T --mp
   8,2 --no-spliced-alignment --directional-mapping -S ./
   temp.sam
```

The performance under single-process execution is illustrated in Figure .

The execution time for runs utilizing multiple processes is illustrated in Figure 6.15.

Considering the large output files generated by the application, disk I/O operations became one of the performance bottlenecks. We addressed this by writing output data to memory filesystem or using pipes to directly pass the data stream to the `samtools` tool, reducing disk I/O latency and accelerating the overall data processing process.

With fewer working threads, the alignment process in *hisat-3n* exhibited near-linear speedup. However, when the number of threads exceeded a certain threshold, thread contention and lock mechanisms introduced significant performance overhead. To mitigate this, we adopted lock-free queues and dedicated input/output threads to achieve lock-free I/O operations.

Further analysis showed that simply increasing the number of working threads did not yield significant speedup once a certain level was reached. Therefore, we introduced a process-level parallelization strategy, which significantly improved performance.

Ultimately, with 48 working threads, we achieved approximately a **13.4x** speedup, significantly enhancing the execution efficiency of the tool.

6.5.5 Overall Optimization Result: 17.6x Speedup

After implementing tool optimizations and workflow enhancements, we achieved a final execution time of **37.2 minutes** for the entire workflow. When compared to the

parallelized dataset's (originally 247.6 minutes), we achieved a **6.66x** speedup. When compared to the single-process baseline for the same dataset, we achieved a **17.6x** speedup.

Detailed execution times for individual tools and their optimization ratios are provided in Figure 6.17 and Table 6.5.

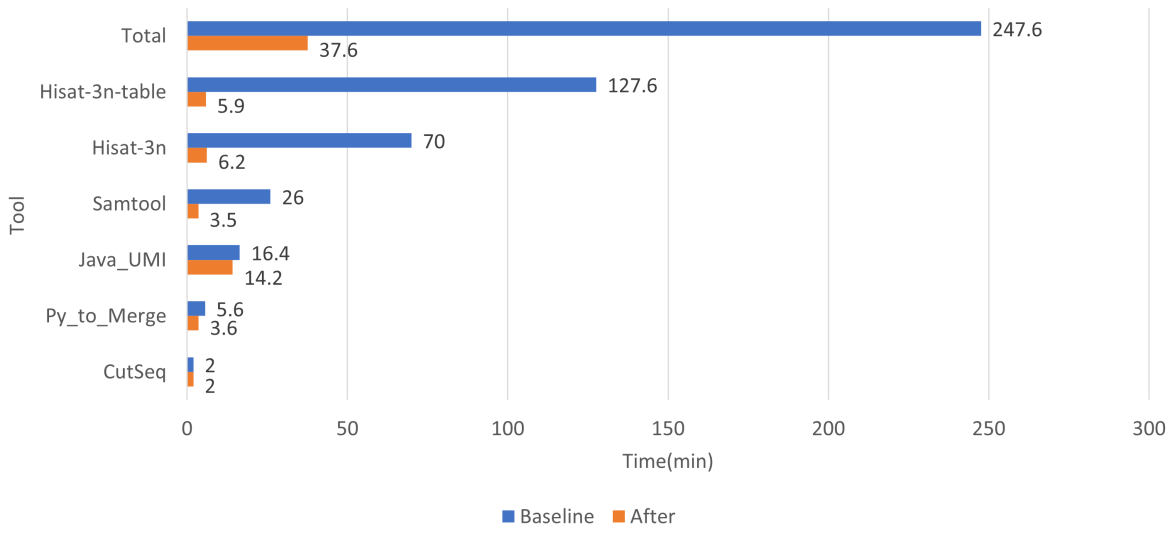

Figure 6.16 Overall Optimization Result of Entire Workflow

Test Case	Baseline	Optimal Result	Speedup Ratio
total	247.6	37.6	6.6x
hisat-3n-table	127.6	5.9	21.6x
hisat-3n	70	6.2	11.3x
samtool	26	3.5	7.4x
java_umi	16.4	14.2	1.2x
py_to_merge	5.6	2.7	2.1x
cut	2	2	1x

Table 6.4 Optimal Results on CPUs of different test cases

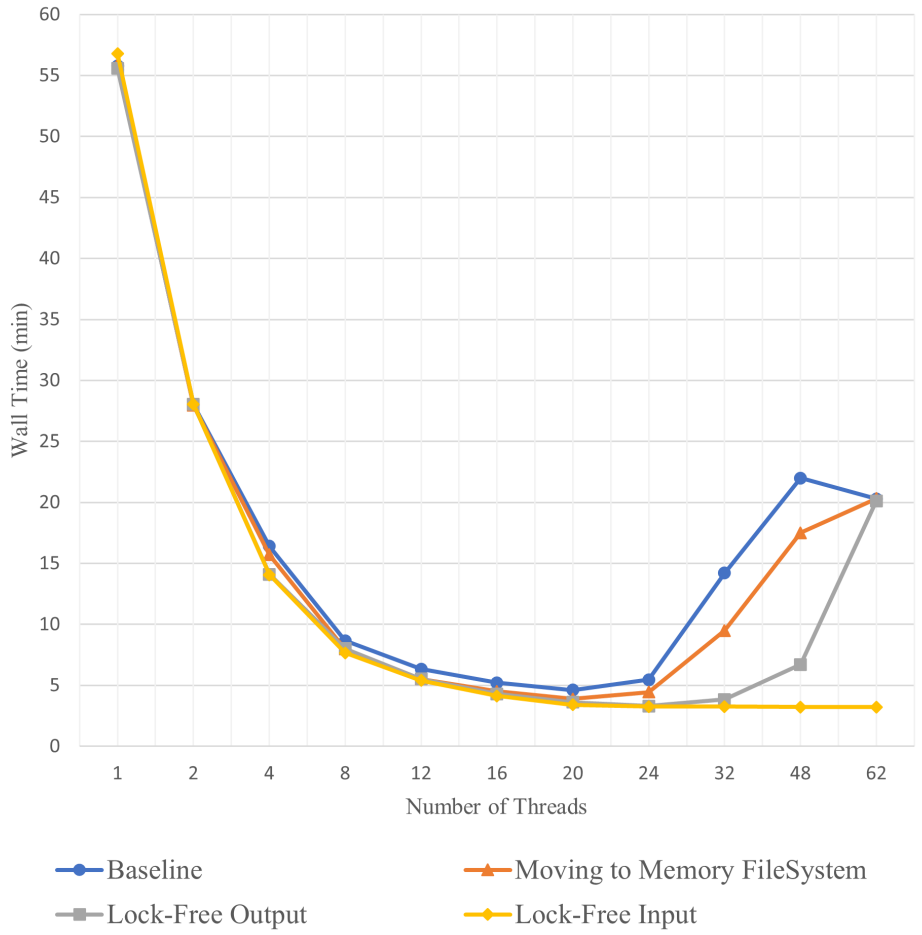

Figure 6.14 Overall SpeedUp of Hisat-3n on Single Process

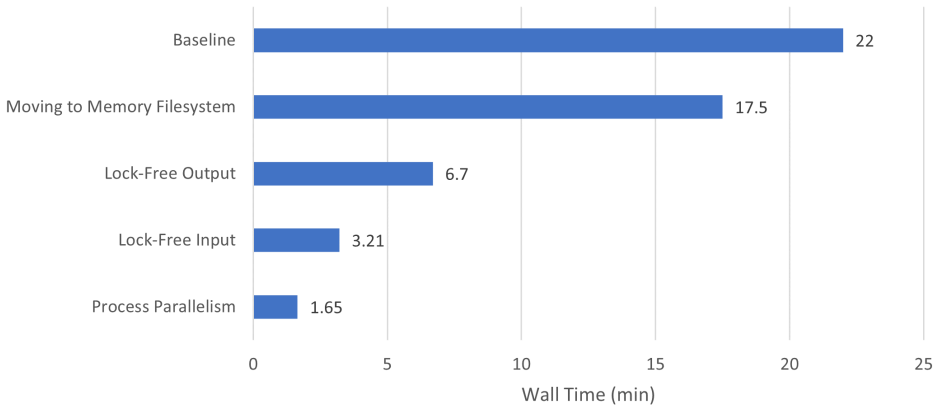

Figure 6.15 Overall SpeedUp of Hisat-3n using Multiple Processes

6.6 Conclusion

The accurate detection of RNA m^5c modifications is pivotal for advancing epigenetic research and clinical applications, yet existing workflows face challenges in computational efficiency and scalability. This study analyze the various components of m^5C-UBSseq, an efficient workflow detecting RNA m^5c modications, and optimizes both the overall workflow and individual tools, implementing optimizations including:

- Parallelizing the independent steps within the workflow;
- Transitioning the working environment to an in-memory filesystem to significantly enhance I/O performance, particularly for large file operations;
- Tuning parameters and buffer sizes of hisat-3n-table to minimize frequent synchronization;
- Utilizing lock-free queues and batch block processing techniques to reduce frequent locking operations and contention;
- Improving loading and output speeds by employing OpenMP to parallelize locally independent parts and deferring string assignments;
- Replacing the standard STL memory allocator with jemalloc to optimize memory management in multi-threaded environments;
- Partitioning the input set to enable process-level parallelism;
- Decoupling dedicated input/output threads and introducing input/output buffers in hisat-3n.

These enhancements collectively reduced the workflows execution time from **651.6 minutes** to **37.2 minutes**, yielding a **17.6x speedup** over the baseline while maintaining high detection accuracy. The optimized workflow demonstrates robust scalability across multi-core architectures, with hisat-3n and hisat-3n-table achieving **13.4x** and **17x** speedups, respectively.

Despite these advancements, opportunities for further improvement remain. For instance, although we have accelerated the loading speed in *hisat-3n-table* by employing lock-free queues and delaying string assignment, the loading speed of the main thread still does not fully meet the throughput demands of all worker threads. Separating the loading functionality from the main thread may provide a more effective solution to this bottleneck. In addition, vectorizing the function `unpack_2b_from_8b` in *hisat-3n* could represent a promising approach to further accelerate computation.

Reference

[1] cameron314, "concurrentqueue," 2024, accessed: 2025-01-28. [Online]. Available: https://github.com/cameron314/concurrentqueue